



SCHEDULING AND LOT STREAMING IN FLOWSHOPS WITH NO-WAIT IN PROCESS

NICHOLAS G. HALL¹, GILBERT LAPORTE², ESAIGNANI SELVARAJAH³, AND
CHELLIAH SRISKANDARAJAH⁴

¹ *The Ohio State University*

² *GERAD and École des Hautes Études Commerciales, Montréal, Canada*

³ *McMaster University, Ontario, Canada*

⁴ *The University of Texas, Dallas*

ABSTRACT

Lot streaming involves splitting a production lot into a number of sublots, in order to allow the overlapping of successive operations, in multi-machine manufacturing systems. In no-wait flowshop scheduling, sublots are necessarily consistent, that is, they remain the same over all machines. The benefits of lot streaming include reductions in lead times and work-in-process, and increases in machine utilization rates. We study the problem of minimizing the makespan in no-wait flowshops producing multiple products with attached setup times, using lot streaming. Our study of the single product problem resolves an open question from the lot streaming literature. The intractable multiple product problem requires finding the optimal number of sublots, sublot sizes, and a product sequence for each machine. We develop a dynamic programming algorithm to generate all the nondominated schedule profiles for each product that are required to formulate the flowshop problem as a generalized traveling salesman problem. This problem is equivalent to a classical traveling salesman problem with a pseudopolynomial number of cities. We develop and computationally test an efficient heuristic for this problem. Our results indicate that solutions can quickly be found for flowshops with up to 10 machines and 50 products. Moreover, the solutions found by our heuristic provide a substantial improvement over previously published results.

KEY WORDS: machine scheduling; lot streaming and batching; no-wait flowshops; generalized traveling salesman problem

1. INTRODUCTION

For the production of medium to high volume products with long setup times, identical units of a single product are often grouped into production *lots*. In traditional batch production systems, a lot is transferred from one machine to the next, only when all the items of the lot are completed. This reduces the time lost to setups. However, it also results in long lead times and high average work-in-process inventory. By using small sublots so that production on consecutive machines can be overlapped, the average work-in-process can be reduced. This splitting of a lot into sublots so that its operations can be overlapped is called *lot streaming* (Baker, 1990; Trietsch and Baker, 1993).

*Correspondence to: Nicholas G. Hall, hall.33@osu.edu.

We study scheduling and lot streaming problems in production environments configured as a flowshop. The objective is the minimization of the schedule completion time, or *makespan*. In a *flowshop* all products follow the same machine sequence, and each product has exactly one operation on each machine. Useful general references on machine scheduling problems include the books by Baker (1974) and Pinedo (1995). Basic results on flowshop problems can be found there and in Monma and Rinnooy Kan (1983).

In the production environment studied here, the processing of sublots is *no-wait* in the sense that each subplot is processed continuously from its start on the first machine to its completion on the last machine, without any interruption on machines and without any waiting time between machines. However, machines may be idle between sublots. Some interest has arisen in no-wait scheduling problems in recent years. This interest is motivated by applications, described for example in Goyal and Sriskandarajah (1988), Hall and Sriskandarajah (1996) and the works cited there, as well as by research questions.

Motivation for studying our flowshop problem comes from a real world problem that arises in an anodizing line, which is a flowshop made up of a series of chemical processing tanks (Hsu and Stein, 1991). The line produces many products, such as pipes, trims, and truck grilles for commercial and automotive industries. The objective is to manufacture the daily requirements of various products in the shortest possible time. In front of the line is a racking area, where products are loaded onto racks prior to chemical processing. The products need to be processed as no-wait for two reasons. First, there is no buffer between the tanks. Second, once a rack exits a tank it must immediately enter the next one, otherwise the products on the rack are ruined by exposure to the atmosphere. Since it is a chemical process, the processing time of a rack, that is, a subplot, in a tank depends on the total surface area. Hence, the processing time is proportional to the number of items in a rack. There is a setup time to change the chemical composition of the tanks when processing is switched from one product to another. This is a no-wait flowshop with lot streaming.

Closely related to this paper are studies of scheduling problems in no-wait flowshops where each product is processed as an individual lot. Gilmore and Gomory (1964) provide an efficient algorithm for minimizing makespan in a two machine no-wait flowshop. Röck (1984) shows that the corresponding problem with three machines is unary NP-hard.

Several studies consider scheduling problems with lot streaming. See Potts and Van Wassenhove (1992) for an early survey. Potts and Baker (1989) study single product lot streaming in a flowshop. They show that, for two and three machine flowshops when the product is infinitely divisible, the optimal solution is obtained by using *consistent sublots*, as defined in Section 2. In the no-wait flowshop considered here, feasibility requires consistent sublots. For flowshops producing multiple products, Potts and Baker (1989) show that the sequencing and lot streaming problems cannot be decomposed. Baker (1995) uses scheduling models with time lags and setup times to analyze the lot streaming of multiple products with equal-sized sublots in two machine flowshops. He also discusses extensions to special cases with more than two machines. Glass, Gupta, and Potts (1994) use a network model to analyze three machine flowshop problems with lot streaming, and then extend their results to three machine openshop problems and two machine jobshop problems. The authors use the algorithm of Gonzalez and Sahni (1976) to solve m machine problems with two equal-sized sublots. Vickson (1995) uses linear programming for the lot streaming and scheduling of multiple products in a two machine flowshop with unlimited buffers. He introduces models for optimal lot streaming and sequencing problems with continuous lot sizes, attached/detached setup times, and/or

sublot transfer times. Also, an algorithm is presented for the integer lot size version. Sriskandarajah and Wagneur (1999) study two machine flowshops with both continuous and integer lot sizes in a no-wait environment.

This paper develops a heuristic for minimizing the makespan in m -machine no-wait flowshops with lot streaming and integer lot sizes. The intractable multiple product problem requires finding the optimal number of sublots, sublot sizes, and a product sequence for each machine. We show that this problem can be formulated as a generalized traveling salesman (GTSP) problem. Thus, one contribution of our work is extending the class of important practical problems that can be solved using the GTSP. An effective heuristic is developed to solve this problem. Second, we resolve an open question from the single product lot streaming literature. Our second contribution is to provide a pseudopolynomial time algorithm for single product lot streaming in no-wait flowshops.

This paper is organized as follows. The notations and assumptions of our model appear in Section 2. Section 3 considers the minimization of makespan in a no-wait flowshop problem with lot streaming. We develop a dynamic programming algorithm that finds optimal sublot sizes, which enables us to show that this problem can be formulated as a GTSP. As a result, we are able to resolve in part the complexity of a single product lot streaming problem discussed in Trietsch and Baker (1993). Section 4 contains a description of a previously published heuristic for the flowshop problem that is used for computational comparisons in our work. In Section 5, we provide extensive computational tests of our procedure and of that earlier heuristic. A discussion about how to extend our model to solve some related practical problems appears in Section 6. Finally, Section 7 contains a conclusion and some suggestions for future research. A related paper by Hall et al. (2002) provides similar results for no-wait openshop problems with lot streaming.

2. NOTATION AND ASSUMPTIONS

Data

$M_1, \dots, M_m$: machines in the shop
 $P_1, \dots, P_N$: products to be manufactured
 p_{ij} : processing time of one unit of product P_j on machine M_i
 n_j : maximum number of sublots available for product P_j
 W_j : number of units of product P_j demanded
 s_{ij} : setup time of product P_j on machine M_i .

Variables

X_{kj} : total number of units of product P_j in sublot k , where $X_{kj} \geq 0$ and integer
 μ_j : optimal number of nonempty sublots for product P_j , where $1 \leq \mu_j \leq n_j$
 σ : sequence of products $P_1, \dots, P_N$
 $P_{\sigma(i)}$: i th product in sequence σ .

Assumptions

1. All products $P_1, \dots, P_N$ are available at time zero.
2. The processing of sublots is no-wait.

3. Each product P_j has a specified maximum allowable number of sublots, n_j . This assumption is motivated by a production environment in which the number of pallets is limited, and each product is allocated a specified number of pallets to transport the sublots.
4. A product's sublots are *consistent*, that is, the subplot sizes remain the same on all machines, as is necessary in no-wait scheduling.
5. There is no limit on the pallet capacity, thus there is no limit on the subplot sizes.
6. The products are not split during processing (that is, no *preemption* is allowed). This means that all the sublots of a product are produced consecutively. For operational convenience, this is usually the case in practice.
7. The processing time of a subplot is proportional to its size, for example, the processing time of the k th subplot of size X_{kj} of product P_j on machine M_i is $X_{kj}p_{ij}$. However, as we discuss in Section 6, processing times that are any nondecreasing function of lot size can be modeled in the way we describe.
8. Switching from one product to another requires a setup on a machine. We assume that the setup times are attached to the products and are independent of the processing sequence of the products on the machine.

3. NO-WAIT FLOWSHOPS

In this section we describe and test a heuristic for the no-wait flowshop problems with lot streaming. In Section 3.1, we discuss the structure of schedules for these problems. Section 3.2 provides an example that shows the difference between no-wait schedules and those that allow waiting. In Section 3.3, we consider scheduling problems with *given* subplot sizes $X_{1j}, \dots, X_{n_jj}$ for each product P_j . In Section 3.4, we consider *simultaneous* lot streaming and scheduling decisions in flowshop problems. Section 3.5 contains a description of our overall algorithm for finding optimal integer subplot sizes for multiple product problems.

3.1. Structure of schedules

In order to motivate the discussion which follows, consider an example where 12 units of a single product must be processed on 3 machines. The per unit processing times are 1, 3, and 2 min on machines M_1 , M_2 , and M_3 , respectively. The given maximum allowable number of sublots is three. If the batch is processed as a single lot, then the makespan is $12(1 + 3 + 2) = 72$ min. However, if the batch is split into three sublots with 2, 6, and 4 units, respectively, then the makespan is only 46 min (see Figure 1).

We illustrate the structure of a schedule, S , for a single product with subplot sizes $X = (X_1, \dots, X_n)$ in an example with three machines. Here we omit the second subscript on X ,

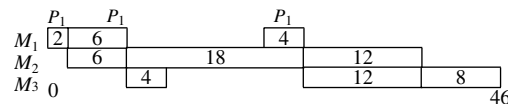


Figure 1. Sublots of a single product in a three machine flowshop: makespan = 46

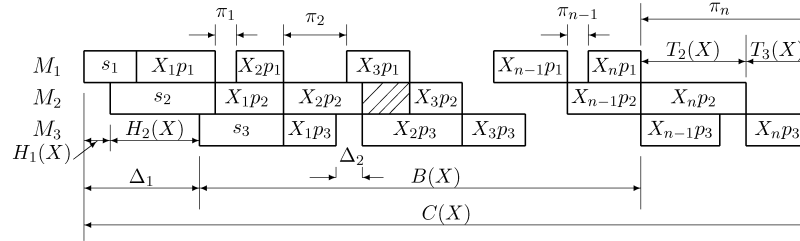


Figure 2. Structure of a single product schedule in a three machine flowshop

since we are considering only a single product. The setup times of the product are attached with the first subplot. Schedule S has five distinct sections: H_1 , H_2 , B , T_2 , and T_3 , as shown in Figure 2. These sections are of lengths $H_1(X)$, $H_2(X)$, $B(X)$, $T_2(X)$, and $T_3(X)$, respectively, and are now described in detail. Let π_i (respectively, Δ_i) denote the idle time on machine M_1 after product P_i (respectively, on machine M_m before product P_i). More discussion of these idle times appears in Sections 3.5 and 4.1, respectively.

H_1 : This is the first section of S , where machines M_2 and M_3 are idle throughout. The length of H_1 is given by: $H_1(X) = \max\{0, s_1 + X_1p_1 - s_2\}$ if $s_3 \leq s_2 + X_1p_2$; and by $H_1(X) = \max\{0, s_1 + X_1p_1 + X_1p_2 - s_3\}$ if $s_3 > s_2 + X_1p_2$.

H_2 : This is the nonnegative time from the start of the setup on M_2 to the start of the setup on M_3 . The length of H_2 is given by $H_2(X) = \max\{0, s_2 + X_1p_2 - s_3\}$.

B : The length of B is given by the time from the start of the setup on M_3 to the finish of the last subplot on M_1 . Thus, $B(X) = C(X) - \max\{0, H_1(X)\} - \max\{0, H_2(X)\} - T_2(X) - T_3(X)$, where $C(X)$ denotes the makespan. As we discuss in Section 3.3, the length of $B(X)$ can be negative; however, this does not affect our formulation or solution procedure.

T_2 : This is the section immediately after B , where machine M_1 is idle. The length of T_2 is given by $T_2(X) = X_np_1$.

T_3 : This is the last section of S , where both machines, M_1 and M_2 , are idle. The length of T_3 is given by $T_3(X) = X_np_3$.

When the subplot sizes $X = (X_1, \dots, X_n)$ change, the profiles defined by H_1 , H_2 , B , T_2 , and T_3 in schedule S also change. H_1 and H_2 are called the *heads*, and T_2 and T_3 are called the *tails*, of the schedule profile. B is called the *body* of the schedule profile. Many schedule profiles are available for a single product. A similar schedule for an m machine flowshop has $(m-1)$ heads, one body, and $(m-1)$ tails. Observe that, in this no-wait environment, minimizing the makespan is equivalent to minimizing the total idle time on the first machine plus the sum of processing times of the last job on all the machines except the first.

3.2. Scheduling multiple products in no-wait flowshops

The following example illustrates the difference between no-wait schedules and those that allow jobs to wait between the machines. Consider an example of a two machine problem with three products P_1, P_2, P_3 , where $n_j = 2$, for $j = 1, 2, 3$. The number of items required for each product is given by: $W_1 = W_2 = W_3 = 2$. The processing times are: $p_{11} = 1, p_{21} = 14, p_{12} = 2, p_{22} = 3, p_{13} = 15, p_{23} = 14$. There is no setup time on the machines. The makespan of the optimal schedule with waiting is 63 as shown in Figure 3, where the sequence of products is

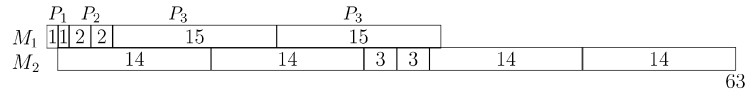


Figure 3. Optimal schedule with waiting: makespan = 63

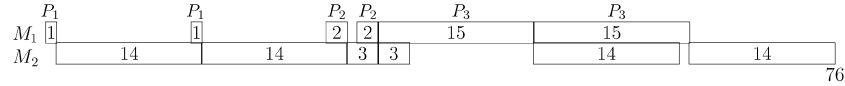


Figure 4. Suboptimal No-wait Schedule: makespan = 76

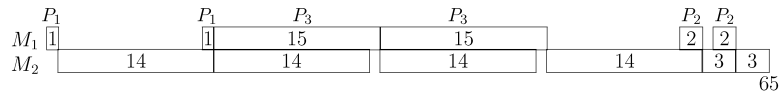


Figure 5. Optimal no-wait schedule: makespan = 65

P_1 , P_2 , P_3 , and each product uses two sublots. If the optimal sequence with waiting is used in the no-wait case, then the makespan is much larger at 76, as shown in Figure 4. The makespan of the optimal no-wait schedule is 65 as shown in Figure 5, where the sequence of products is P_1 , P_3 , P_2 , and each product uses two sublots.

3.3. Scheduling with given subplot sizes

For each product P_j , let the given subplot sizes $X_{1j}, \dots, X_{n_jj}$, define a unique profile. Assume that the N products, $P_1, \dots, P_N$, in the flowshop are scheduled in the order $P_{\sigma(1)}, \dots, P_{\sigma(N)}$, where $P_{\sigma(r)}$ is the r th product in sequence σ . Note that, from Assumption 6, all the sublots of a given product are processed consecutively. We need to find an optimal product sequence σ^* that, for a given vector of subplot sizes for each product, minimizes the makespan. For any number of machines, this problem can be formulated as a classical traveling salesman problem (TSP), as described below. See Lawler et al. (1985) for background on the TSP.

Consider the three machine example shown in Figure 6. Since the subplot sizes are given, the lengths of H_{1j} , H_{2j} , B_j , T_{2j} , and T_{3j} are well defined. We define a dummy product, 0, that starts and ends the schedule, respectively, where $H_{1\sigma(i)} = H_{2\sigma(i)} = B_{\sigma(i)} = T_{2\sigma(i)} = T_{3\sigma(i)} = 0$ for $i = 0$, as in Figure 6. The reason for introducing this dummy product is to convert the makespan problem into an equivalent TSP problem whose sequence is cyclical (Hall and Sriskandarajah, 1996). The makespan, F_σ , can be written as:

$$F_\sigma = \sum_{i=1}^{N+1} \max\{H_{1\sigma(i)} + H_{2\sigma(i)}, T_{2\sigma(i-1)} + H_{2\sigma(i)}, T_{2\sigma(i-1)} + T_{3\sigma(i-1)}\} + \frac{B_{\sigma(i-1)}}{2} + \frac{B_{\sigma(i)}}{2}$$

or

$$F_\sigma = \sum_{i=1}^{N+1} g_{\sigma(i-1)\sigma(i)} + \frac{B_{\sigma(i-1)}}{2} + \frac{B_{\sigma(i)}}{2} = \sum_{i=1}^{N+1} d_{\sigma(i-1)\sigma(i)}$$

where $d_{\sigma(i-1)\sigma(i)} = g_{\sigma(i-1)\sigma(i)} + B_{\sigma(i-1)}/2 + B_{\sigma(i)}/2$, $g_{\sigma(i-1)\sigma(i)} = \max\{H_{1\sigma(i)} + H_{2\sigma(i)}, T_{2\sigma(i-1)} + H_{2\sigma(i)}, T_{2\sigma(i-1)} + T_{3\sigma(i-1)}\}$ and $N+1=0$. Now, $g_{\sigma(i-1)\sigma(i)}$ is the time from the finish of the body of product $P_{\sigma(i-1)}$ to the start of the body of product $P_{\sigma(i)}$, and $d_{\sigma(i-1)\sigma(i)}$ may be

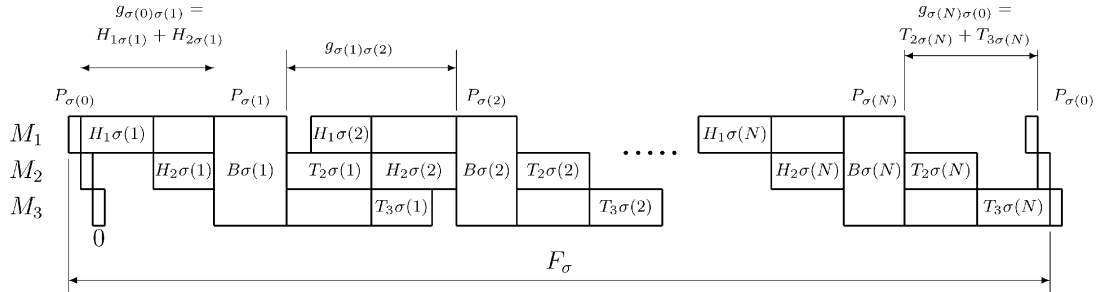


Figure 6. Example flowshop schedule with three machines

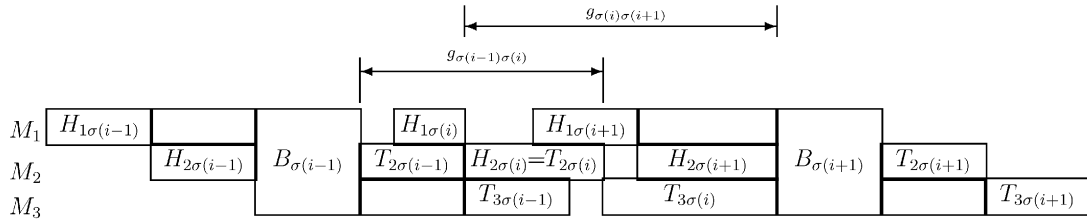


Figure 7. Schedule with a negative body in a three machine flowshop

interpreted as the distance from city $i - 1$ to city i in a TSP formulation. Therefore, the problem with given subplot sizes can be solved by solving the corresponding TSP.

Using a schedule for a three-machine flowshop, we now illustrate in Figure 7 how a product can have a negative body B . Here, product $P_{\sigma(i)}$ has a single subplot in a three machine flowshop, therefore $H_{2\sigma(i)}$ and $T_{2\sigma(i)}$ mean the same thing. Moreover, since the lengths $g_{\sigma(i-1)\sigma(i)}$ and $g_{\sigma(i)\sigma(i+1)}$ overlap by the amount $H_{2\sigma(i)} = T_{2\sigma(i)}$, that length is added twice to the total GTSP distance. By definition, we have a negative body for product $P_{\sigma(i)}$, given by $B_{\sigma(i)} = -H_{2\sigma(i)}$. This negative body is permitted in our GTSP formulation, and therefore does not affect Algorithm GlobalFlow.

3.4. Simultaneous lot streaming and scheduling

We consider the problem of simultaneously sequencing and lot streaming multiple products with integer-sized sublots. Equivalently, this problem requires simultaneously selecting a sequence of products and a profile for each product. This problem can be modeled as a GTSP. The GTSP is a generalization of the classical TSP, and is usually described as having a large number of cities which are divided into countries. A solution to the GTSP is a tour that visits exactly one city in each country. Here, each product represents a country. The possible profiles for that product represent different cities within that country. Therefore, the choice of a profile for that product can be modeled as the choice of a city within that country. Thus, scheduling all the products and selecting a profile for each is equivalent to visiting exactly one city in each country. The GTSPs under consideration in this study possess an asymmetric distance structure. There are three basic algorithmic approaches for the asymmetric GTSP. Regarding the first two of these, the exact algorithm by Noon and Bean (1991) and the transformation into a TSP by

Noon and Bean (1993) cannot be used here, due to the limitation on the number of cities which those approaches can handle.

A more practical approach for large problem instances, such as those defined by our flowshop problems, is to solve the GTSP directly by means of a heuristic. The method we develop builds on an algorithm proposed by Gendreau, Laporte, and Simchi-Levi (1991). An assignment problem is first solved on the countries, using minimum inter-country distances. If the resulting solution is feasible for the GTSP, then it is also optimal. Otherwise, the assignment solution is transformed into a single tour over all countries by means of a patching technique (Karp, 1979). We also apply a two phase improvement procedure. In the first phase, we use tabu search (Glover and Laguna, 1997) where neighboring solutions are defined by considering all swaps of consecutive countries in the current tour, and then determining for each country, in a greedy fashion, the best city to be visited so as to yield a minimal length tour. The best move is then implemented, even if this causes the current objective function value to deteriorate. To avoid cycling, the reverse of an implemented move is declared tabu for 10 iterations. In the second phase we use a genetic algorithm where the tours obtained in the first phase are used as the initial population. We use a mutation probability of 0.1 and a crossover probability of 0.8. The population sizes are varied according to the number of countries in the problem. The procedure is terminated after 2,000 iterations of the genetic algorithm.

3.5. Algorithm

We now describe our overall algorithm, GlobalFlow, for the flowshop problem. For each product, all optimal profiles are found for all possible head and tail lengths. Note that the head and tail lengths depend only on the size of the first and last sublots of the product (see Figure 2). We define a vector $v = (v_1, v_4)$, where the integer v_1 (respectively, v_4) represents the size of the first (respectively, last) subplot, and $v_1, v_4 \geq 0$.

In order to find an optimal profile, that is, optimal integer subplot sizes, for a given (v_1, v_4) and a given product P_j , a dynamic programming subroutine, $DP_F^v(j)$, is called in Algorithm GlobalFlow. Subroutine $DP_F^v(j)$ chooses the subplot sizes such that the total idle time on machine M_1 , represented by $\sum_{i=1}^n \pi_i$ in Figure 2, is minimized. Thus, it generates all the nondominated profiles for all the products, as required for our GTSP formulation. Once all the profiles for each product are found, the resulting GTSP is solved heuristically to find a profile for each product and schedule the products simultaneously, as now described.

Algorithm GlobalFlow

Initialize $j = 1$.

While $j \leq N$ do

For all $\{v = (v_1, v_4) \mid v_1, v_4 \geq 0, v_1 + v_4 \leq W_j\}$

If $\min\{v_1, v_4\} \geq 1$, then do

Call $DP_F^v(j)$.

Using the optimal subplot sizes found, define a product profile which corresponds to a city in country j .

End (if)

End (for)

Set $j = j + 1$.

End (while)

Apply the heuristic described in Section 3.4 to the GTSP, and output the solution.

It remains to define the dynamic programming Subroutine $DP_F^v(j)$. In order to describe this algorithm for m machine flowshops, we define an integer $Y_k \geq 0$, for $k = 1, \dots, n_j$. The optimal μ_j value which can be determined from $DP_F^v(j)$ is the number of strictly positive Y_k values. Note that the second index of p_{ij} is omitted, since all calculations are for product P_j .

Subroutine $DP_F^v(j)$

Optimal value function: Let $f_k(Y_k, w, r)$ denote the total minimum idle time on M_1 for the partial schedule formed by sublots $Y_{k+1}, \dots, Y_{n_j}$, given that the k th subplot has size Y_k and that there are w units and r sublots left to allocate.

Boundary conditions: $f_t(Y_t, v_4, r) = \max\{0, p_2 Y_t - p_1 v_4, p_2 Y_t + p_3 Y_t - p_1 v_4 - p_2 v_4, p_2 Y_t + p_3 Y_t + p_4 Y_t - p_1 v_4 - p_2 v_4 - p_3 v_4, \dots, p_2 Y_t + p_3 Y_t + p_4 Y_t + \dots + p_m Y_t - p_1 v_4 - p_2 v_4 - p_3 v_4 - \dots - p_{m-1} v_4\}$ for $1 \leq r \leq n_j - 1, t \in \{1, \dots, n_j - 1\}$; $f_0(\cdot, \cdot, \cdot) = +\infty$ otherwise

Optimal solution: $f_0(v_1, W_j - v_1, n_j - 1)$.

Recurrence relation: $f_k(Y_k, w, r) = \min_{1 \leq Y_{k+1} \leq w} \{\max\{0, p_2 Y_k - p_1 Y_{k+1}, p_2 Y_k + p_3 Y_k - p_1 Y_{k+1} - p_2 Y_{k+1}, p_2 Y_k + p_3 Y_k + p_4 Y_k - p_1 Y_{k+1} - p_2 Y_{k+1} - p_3 Y_{k+1}, \dots, p_2 Y_k + p_3 Y_k + p_4 Y_k + \dots + p_m Y_k - p_1 Y_{k+1} - p_2 Y_{k+1} - p_3 Y_{k+1} - \dots - p_{m-1} Y_{k+1}\} + f_{k+1}(Y_{k+1}, w - Y_{k+1}, r - 1)\}$.

Note that the discrete lot size (DV) version of the single product m machine no-wait (NW) flowshop problem with consistent (C) sublots and intermittent idling (II) on machines is in the class NP, but has open complexity in the literature. We denote this problem $m | C | II | DV | NW$, which is consistent with the notation used by Trietsch and Baker (1993). Using Algorithm GlobalFlow, we are able to resolve this complexity question partially.

Theorem 3.1. Problem $m | C | II | DV | NW$ is optimally solvable in pseudopolynomial time.

Proof. The proof follows from Algorithm GlobalFlow, which calls Subroutine $DP_O^v(j)$ a total of $O(W_j^2)$ times for given j . We now consider the running time of Subroutine $DP_F^v(j)$. The number of possible vectors v and r is $O(n_j)$ each. The number of possible values of w and Y_k is $O(W_j)$ each. Consequently, the recurrence relation is computed $O(n_j^2 W_j^2)$ times. Each such computation requires $O(W_j)$ comparisons. Thus, the overall computation time of GlobalFlow is $O\left(\sum_{j=1}^N n_j^2 W_j^5\right)$. Therefore, the overall computation time of Algorithm GlobalFlow when applied to the single product problem is pseudopolynomial. ■

4. PREVIOUS STUDY

In this section, we discuss some closely related heuristics in the literature. Section 4.1 considers the lot streaming of a single product with fractional subplot sizes. The integer subplot size case is considered in Section 4.2. Section 4.3 considers the lot streaming and sequencing of multiple products.

4.1. Lot streaming of a single product: fractional sublots

We first illustrate the lot streaming methodology of Sriskandarajah and Wagneur (1999) (see Figure 2 for an example with $m = 3$). Let Δ_i be the idle time on the *last* machine (M_3) before the

processing of subplot i begins. Since we are considering a single product, the index j denoting product type is dropped. With these definitions, the makespan $C(X)$ can be written as follows (see Figure 2):

$$C(X) = s_3 + p_3 W + \sum_{i=1}^n \Delta_i \quad (1)$$

Note that in (1), the makespan is given as a function of the idle times $\Delta_1, \dots, \Delta_n$ on the last machine M_3 . Let $Z = \sum_{i=1}^n \Delta_i$, where $\Delta_1 = \max\{0, s_2 + X_1 p_2 - s_3, X_1 p_1 + X_1 p_2 + s_1 - s_3\}$, and $\Delta_i = \max\{0, X_i p_2 - X_{i-1} p_3, X_i p_1 + X_i p_2 - X_{i-1} p_2 - X_{i-1} p_3\}$, for $i = 2, \dots, n$. The minimum makespan problem (1) can then be formulated as the following linear program:

$$\begin{aligned} \min Z &= \sum_{i=1}^n \Delta_i \\ \text{subject to} \\ \sum_{i=1}^n X_i &= W \\ \Delta_1 &\geq s_2 + X_1 p_2 - s_3 \\ \Delta_1 &\geq s_1 + X_1 p_1 + X_1 p_2 - s_3 \\ \Delta_i &\geq X_i p_2 - X_{i-1} p_3 \quad i = 2, \dots, n \\ \Delta_i &\geq X_i p_1 + X_i p_2 - X_{i-1} p_2 - X_{i-1} p_3 \quad i = 2, \dots, n \\ X_i &\geq 0 \quad i = 1, \dots, n \\ \Delta_i &\geq 0 \quad i = 1, \dots, n \end{aligned}$$

This formulation can be generalized to any value of m . The solution to the linear program provides fractional subplot sizes $X_1, \dots, X_n$.

4.2. Lot streaming of a single product: integer sublots

Kumar, Bagchi, and Sriskandarajah (2000) consider the problem of finding optimal integer subplot sizes $Y_1, \dots, Y_n$ for a single product. They use as input the fractional subplot sizes $X_1, \dots, X_n$ found in Section 4.1. They then apply Heuristic S&W of Sriskandarajah and Wagneur (1999), which is now described. This heuristic starts by assigning to each lot the smallest integer number of units that exceeds the fractional allocation. Since the total number of units assigned is more than required, the assignments to sublots are then reduced one by one so as to minimize the maximum deviation of any integer value Y_i from the corresponding optimal fractional value X_i .

Heuristic S&W

Set $W_o = 0$, $W_1 = W$ and $\Gamma = \emptyset$

For $i = 1$ to n do

$$Y_i = \lfloor X_i \rfloor + 1$$

$$W_o = W_o + Y_i$$

End (for)

```

Set  $W_o = W_o - W_1$ 
Let  $\Gamma = \{i \mid Y_i > 1\}$ 
While  $W_o > 0$  do
    Find  $d_i = Y_i - X_i$ , for  $i \in \Gamma$ 
    Find  $r$  such that  $d_r = \max_{i \in \Gamma} \{d_i\}$ 
    Set  $Y_r = Y_r - 1$ 
    If  $Y_r = 1$ , then set  $\Gamma = \Gamma - \{r\}$ 
    Set  $W_o = W_o - 1$ 
End (while).

```

4.3. Sequencing and lot streaming: multiple products and integer sublots

To solve the multiple product problem, Kumar, Bagchi, and Sriskandarajah (2000) propose the following heuristic approach.

Heuristic MHEU

- Step 1:* Given a specified number of sublots for each product, find fractional sublot sizes using linear programming, and then round these using Heuristic S&W.
- Step 2:* Given these integer solutions for the sublot sizes, compute the resulting product profiles.
- Step 3:* Obtain the product sequence by solving the resulting TSP using the algorithm of Carpaneto and Toth (1980).

5. COMPUTATIONAL RESULTS

As a preliminary step, we evaluated the performance of the GTSP heuristic. To do so, we ran tests on 60 small GTSP instances, using the transformation into a TSP of Noon and Bean (1993) and the optimal algorithm of Carpaneto and Toth (1980) to provide comparisons with results from the GTSP heuristic. Our data set consisted of 20 instances with 3 products and up to 70 profiles each, 20 instances with 4 products and up to 40 profiles each, and 20 instances with 5 products and up to 10 profiles each. For all 60 instances, the GTSP heuristic provides an optimal solution, which is a clear indication of its accuracy for small problem instances.

To evaluate the performance of Algorithm GlobalFlow, we tested it on 360 randomly generated problem instances. Three types of problem instances, I_1 , I_2 , and I_3 , are studied. For all problem instances tested, the processing times p_{ij} are generated randomly from a uniform distribution over the interval $1 \leq p_{ij} \leq 5$. The maximum allowable number of sublots for each product, n_j , is generated randomly from a uniform distribution over the interval $1 \leq n_j \leq 5$. The total number of units demanded for each product, W_j , is generated randomly from a uniform distribution over the interval $2n_j \leq W_j \leq 7n_j$. The setup times are generated randomly as follows: all the setup times are zero ($s_{ij} = 0$) for I_1 , uniform integer over $1 \leq s_{ij} \leq 5$ for I_2 , and uniform integer over $1 \leq s_{ij} \leq 10$ for I_3 .

Each line in Tables I–III shows mean results over ten randomly generated problem instances. Column 3 shows the mean total number of profiles over all products, rounded to the nearest integer. Columns 4 and 6 show the mean makespan values rounded to the nearest integer, for Algorithm GlobalFlow and Heuristic MHEU, respectively. Column 5 shows the total CPU

Table I. Performance of GlobalFlow for three machine flowshop problems

Instance type	Number of products	Number of profiles	Algorithm Global Flow		MHEU $C_{\max}$	Impr.	% impr.
			$C_{\max}$	CPU(s)			
I_1	10	77	600	19	648	48	8.00
I_1	15	101	852	30	902	50	5.87
I_1	25	194	1516	86	1644	128	8.44
I_1	50	397	3039	416	3208	169	5.56
I_2	10	82	639	19	694	55	8.61
I_2	15	112	917	32	999	82	8.94
I_2	25	192	1530	86	1639	109	7.12
I_2	50	398	3227	423	3386	159	4.93
I_3	10	67	651	18	686	35	5.38
I_3	15	107	1001	30	1074	73	7.29
I_3	25	176	1511	87	1611	100	6.62
I_3	50	383	3270	417	3423	153	4.68

Table II. Performance of GlobalFlow for five machine flowshop problems

Instance type	Number of products	Number of profiles	Algorithm Global Flow		MHEU $C_{\max}$	Impr.	% impr.
			$C_{\max}$	CPU(s)			
I_1	10	168	711	22	776	65	9.14
I_1	15	324	1055	38	1144	89	8.44
I_1	25	388	1605	96	1715	110	6.85
I_1	50	1035	3462	478	3679	217	6.27
I_2	10	212	727	23	795	68	9.35
I_2	15	266	1110	36	1195	85	7.66
I_2	25	536	1741	105	1889	148	8.50
I_2	50	1011	3527	471	3735	208	5.90
I_3	10	205	735	22	796	61	8.30
I_3	15	359	1214	39	1310	96	7.91
I_3	25	497	1867	101	2023	156	8.36
I_3	50	1052	3567	484	3796	229	6.42

time taken by Algorithm GlobalFlow and the GTSP, and Columns 7 and 8 show the improvement and percentage improvement, respectively, provided by our approach relative to MHEU. All algorithms are coded in C, compiled using the ANSI C compiler and run on a 143 MHz Sun workstation. As expected, the CPU times vary with the number of products and the number of profiles. However, they do not vary significantly with the number of machines in the flowshop, because the time required to generate the distance matrix and run $DP_F^v(j)$ is small relative to the time required by the GTSP heuristic described in Section 3.4. The CPU time required for Heuristic MHEU is negligible. The cost savings provided by the tabu search and genetic algorithm steps are summarized in Table IV. Each entry x, y denotes a mean saving of $x\%$ from the tabu search and an additional $y\%$ from the genetic algorithm over 40 problem instances.

Table III. Performance of GlobalFlow for ten machine flowshop problems

Instance type	Number of products	Number of profiles	Algorithm Global Flow		MHEU $C_{\max}$	Impr.	% impr.
			$C_{\max}$	CPU(s)			
I_1	10	214	848	23	931	83	9.79
I_1	15	321	1330	37	1435	105	7.89
I_1	25	519	2127	101	2280	153	7.19
I_1	50	980	4163	465	4364	201	4.83
I_2	10	222	902	23	985	83	9.20
I_2	15	355	1403	38	1514	111	7.91
I_2	25	458	2048	100	2193	145	7.08
I_2	50	1102	4443	479	4595	152	3.42
I_3	10	178	931	22	994	63	6.77
I_3	15	287	1372	36	1466	94	6.85
I_3	25	500	2271	100	2419	148	6.52
I_3	50	989	4482	461	4640	158	3.53

Table IV. Percentage cost savings from tabu search and genetic algorithm

Type	m		
	3	5	10
I_1	1.55, 1.95	0.94, 4.13	1.57, 3.97
I_2	1.17, 2.23	1.13, 3.20	1.17, 4.58
I_3	1.28, 1.78	1.40, 3.12	1.21, 4.10

The results in Tables I–III show that our approach provides significant and consistent improvements over Heuristic MHEU, although with greater computational effort. These improvements tend to be larger for problem instances with up to 25 products. The relatively small improvements for the instances with 50 products can be explained by the less accurate performance of the GTSP for larger numbers of countries. Even so, our results represent a substantial improvement over those provided by Heuristic MHEU. The results in Table IV indicate that both the tabu search and genetic algorithm steps provide useful cost savings.

6. EXTENSIONS

In this section, we discuss some possible extensions of the above model for flowshop problems to allow for various practical considerations, including sublots with limited pallet capacity, subplot transfer times between machines, detached and sequence dependent setup times, and general processing times.

(1) *Limited pallet capacity.* The assumption in the paper is that pallets have infinite capacity. Algorithm GlobalFlow can still be used in the case of finite capacity pallets, by restricting $DP_F^v(j)$ to assign items to sublots only up to the given capacity. This restriction reduces the number of possible profiles for a product. Consequently, instances of a given size can typically be solved faster than similar instances of the original problem.

2. *Sublot transfer times.* Here, we consider the fact that transferring sublots between machines takes time. Let δ denote the transfer time from an input stage to the first machine, the transfer time between adjacent machines, and the transfer time from the last machine to an output stage. Algorithm GlobalFlow can take these sublot transfer times into account, as we now describe. Consider a three-machine instance without the transfer times, that is, $\delta = 0$. The idle time on M_1 between the first and second sublots is $\pi_1 = \max\{0, X_1 p_2 - X_2 p_1, X_1 p_2 + X_1 p_3 - X_2 p_1 - X_2 p_2\}$ (see Figure 2). If $\delta > 0$, then we add this transfer time to the processing times of all sublots. Thus, the new idle time is $\pi'_1 = \max\{0, (X_1 p_2 + \delta) - (X_2 p_1 + \delta), (X_1 p_2 + \delta) + (X_1 p_3 + \delta) - (X_2 p_1 + \delta) - (X_2 p_2 + \delta)\}$. Note that $\pi'_1 = \pi_1$. Thus, the idle times between sublots on machine M_1 are not affected by the transfer times. Similarly, it can be shown that the same property holds for idle times between products. Hence, the solution for $\delta = 0$ found by GlobalFlow can still be used. More generally, machine dependent transfer times, for example, where δ_i represents the transfer time between machines M_i and M_{i+1} , can also be modeled in the same way.

3. *Detached setup times.* Algorithm GlobalFlow is designed for attached setup times. We now show that a schedule for detached setup times is the same as a schedule for attached setup times. A detached setup can be performed on a machine as soon as the machine becomes available, even without the physical presence of the arriving sublot of a product. Thus, there exists an optimal schedule in which a setup on a machine starts as soon as the previous product on that machine is completed. Consequently, the setup times of a product P_j can be attached to the end of the last sublot of the previous product P_i in the sequence. This means that there is no change in the distance matrix d_{ij} or in the optimal schedule as a result of the detached setup times.

4. *Sequence dependent setup times.* Sequence dependent setup times can also be included in the GTSP distance matrix and Algorithm GlobalFlow can still be used. The changeover times between products in some production environments involve teardown (i.e., in the anodizing line, draining the old solution) followed by setup (to continue the example, adding the new solution). Note that the teardown time belongs to the previous product, whereas the setup time belongs to the next product. This type of separable changeover time can also be included in our model with an appropriately adjusted GTSP distance matrix, and Algorithm GlobalFlow can still be applied. However, the inclusion of sequence dependent setup times may change the optimal schedule.

5. *General processing times.* We assume throughout the paper that the processing time of a sublot is a linear function of the sublot size X . However, if the processing time is any nondecreasing function $f(X)$, then Subroutine $DP_F^v(j)$ can still be used to find optimal sublot sizes, and thus to obtain an optimal profile, for given sizes of the first and last sublots.

7. CONCLUDING REMARKS

This paper describes a practically useful heuristic for minimizing the makespan in no-wait flowshops with lot streaming. We show that this problem can be formulated as a GTSP. Since the GTSP can be reformulated as the widely studied classical TSP (Noon and Bean, 1993), many optimal and heuristic solution procedures are available. Thus, one contribution of our work is extending the class of important practical problems that can be solved using the TSP. Second, we resolve an open question from the single product lot streaming literature. Our main contribution, however, is to provide an effective customized heuristic for the multiple product no-wait processing and lot streaming flowshop problem, which is formally intractable. Our

results are particularly interesting in view of a study by Abadi, Hall, and Sriskandarajah (2000) on cycle time minimization in blocking flowshops. A related paper by Hall et al. (2002) describes the design and testing of a heuristic for two machine no-wait openshops with lot streaming.

ACKNOWLEDGEMENTS

This research was supported in part by the National Science Foundation (grant number DMI-9821033), the Summer Fellowship Program (Fisher College of Business, The Ohio State University), the Natural Sciences and Engineering Research Council of Canada (grant numbers OGP0104900 and OGP0039682), and the University of Texas at Dallas.

REFERENCES

- Abadi, I. N. K., N. G. Hall, and C. Sriskandarajah, "Minimizing cycle time in a blocking flowshop," *Oper. Res.*, **48**, 177–180 (2000).
- Baker, K. R., *Introduction to Sequencing and Scheduling*, Addison-Wesley, Reading, MA, 1974.
- Baker, K. R., "Solution procedures for the lot-streaming problem," *Decision Sci.*, **21**, 475–491 (1990).
- Baker, K. R., "Lot streaming in the two machine flow shop with setup times," *Ann. Oper. Res.*, **57**, 1–11 (1995).
- Carpaneto, G. and P. Toth, "Some new branching and bounding criteria for the asymmetric traveling salesman problem," *Manage. Sci.*, **26**, 736–743 (1980).
- Gendreau, M., and G. Laporte, D. Simchi-Levi, "A degree relaxation algorithm for the asymmetric generalized traveling salesman problem," Publication CRT-800, Centre for Research on Transportation, Montreal, 1991.
- Gilmore, P. C. and R. E. Gomory, "Sequencing a one-state variable machine: A solvable case of the travelling salesman problem," *Oper. Res.*, **12**, 665–679 (1964).
- Glass, C. A., J. N. D. Gupta, C. N. Potts, "Lot streaming in three-stage process," *Eur. J. Oper. Res.*, **75**, 378–394 (1994).
- Glover, F. and M. Laguna, *Tabu Search*, Kluwer, Boston, 1997.
- Gonzalez, T. and S. Sahni, "Openshop scheduling to minimize finish time," *J. Assoc. Comput. Mach.*, **23**, 665–679 (1976).
- Goyal, S. K. and C. Sriskandarajah, "No-wait shop scheduling: Computational complexity and approximate algorithms," *Opsearch*, **25**, 220–244 (1988).
- Hall, N. G., G. Laporte, E. Selvarajah, C. Sriskandarajah, "Scheduling and lot streaming in openshops with no-wait in process," Working Paper, Fisher College of Business, The Ohio State University, Columbus, Ohio, submitted for publication, December, 2002.
- Hall, N. G. and C. Sriskandarajah, "A survey of machine scheduling problems with blocking and no-wait in process," *Oper. Res.*, **44**, 510–525 (1996).
- Hsu, E. and M. Stein, "Anodize load scheduling for diamond aluminum," *Undergraduate Thesis*, Department of Industrial Engineering, University of Toronto, Canada, 1991.
- Karp, R. M., "A patching algorithm for the nonsymmetric traveling-salesman problem," *SIAM J. Comput.*, **8**, 561–573 (1979).
- Kumar, S., T. P. Bagchi, C. Sriskandarajah, "Lot streaming and scheduling heuristics for m -machine no-wait flowshops," *Comput. Indust. Eng.*, **38**, 149–172 (2000).
- Lawler, E. L., J. K. Lenstra, and A. H. G. Rinnooy Kan, D. B. Shmoys, *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*, Wiley, Chichester, UK, 1985.
- Monma, C. L. and A. H. G. Rinnooy Kan, "A concise survey of efficiently solvable special cases of the permutation flow-shop problem," *RAIRO Recherche Opérationnelle*, **17**, 105–119 (1983).
- Noon, C. E. and J. C. Bean, "A Lagrangian based approach for the asymmetric generalized traveling salesman problem," *Oper. Res.*, **39**, 623–632 (1991).
- Noon, C. E. and J. C. Bean, "An efficient transformation of the generalized traveling salesman problem," *INFOR*, **31**, 39–44 (1993).
- Pinedo, M., *Scheduling: Theory, Algorithms, and Systems*, Prentice-Hall, Englewood Cliffs, NJ, 1995.
- Potts, C. N. and K. R. Baker, "Flowshop scheduling with lot streaming," *Oper. Res. Lett.*, **8**, 297–303 (1989).

- Potts, C. N. and L. N. Van Wassenhove, "Integrating scheduling with batching and lot-sizing: A review of algorithms and complexity," *J. Oper. Res. Soc.*, **43**, 395–406 (1992).
- Röck, H., "The three machine no-wait flowshop problem is NP-complete," *J. Assoc. Comput. Mach.*, **31**, 336–345 (1984).
- Sriskandarajah, C. and E. Wagneur, "Lot streaming and scheduling multiple products in two-machine no-wait flowshops," *IIE Trans.*, **31**, 695–707 (1999).
- Trietsch, D. and K. R. Baker, "Basic techniques for lot streaming," *Oper. Res.*, **41**, 1065–1076 (1993).
- Vickson, R. G., "Optimal lot streaming for multiple products in a two-machine flowshop," *Eur. J. Oper. Res.*, **85**, 556–575 (1995).